



SYLLABUS

Planificación del proceso de enseñanza-aprendizaje

I. Información básica de la asignatura

Universidad	Universidad Americana (UAM)
Sede	Managua
Unidad académica	Facultad de Ingeniería y Arquitectura
Carrera	Ingeniería en Sistemas de Información
Nombre y código de la asignatura	CNU0225 – Programación Estructurada
Horas de trabajo independiente	45
Horas presenciales	90
Horas prácticas	70
Total de horas	135
Número de créditos	3
Requisito	CNU0125 – Introducción a la Programación
Semestre	2
Nombre del profesor	M.Sc. Armando J. López L.
Contacto del profesor	armando.lopez@uamv.edu.ni
Horario	Lun, Mie, Vie 7:00 AM – 8:50 AM
Grupo	4
Horario de consulta	Asincrónicamente, a través del foro <i>Consultas al docente</i> , en UAM-Virtual

II. Descripción de la asignatura

La asignatura Programación Estructurada constituye un pilar fundamental para los estudiantes de Ingeniería en Sistemas. Sirve como puente entre los conocimientos iniciales de programación y los conceptos avanzados de la programación orientada a objetos. Se centra en la comprensión y aplicación de principios estructurados para el desarrollo de software eficiente, modular y escalable.

Los estudiantes desarrollarán habilidades clave como el análisis de problemas, el diseño de algoritmos, la implementación de estructuras de datos estáticas y dinámicas, así como el uso adecuado de funciones y manejo de errores. Todo ello orientado a fomentar el pensamiento lógico, crítico y creativo, con énfasis en valores como la responsabilidad, la ética profesional y la sostenibilidad.

Como parte del proceso de aprendizaje, los estudiantes utilizarán Python, un lenguaje de programación de alto nivel, ampliamente difundido y de gran relevancia en el contexto tecnológico actual. Su incorporación permitirá llevar a la práctica los principios de la programación estructurada mediante la construcción de soluciones algorítmicas, el desarrollo de programas y la manipulación de estructuras de datos, al mismo tiempo que favorece la adquisición de competencias transferibles a ámbitos de aplicación contemporáneos como el desarrollo de software, la automatización de procesos, el análisis de datos, la inteligencia artificial y otros campos emergentes de la tecnología.



III. Objetivos de aprendizaje e indicadores de logros

Objetivos de Aprendizaje de Asignatura	Indicadores de logro del proceso de aprendizaje (IL)
1. Aplica técnicas de validaciones y manejo de errores de forma eficiente en un lenguaje de programación con el objetivo de desarrollar aplicaciones robustas y fáciles de mantener.	IL1.1. Identifica y clasifica errores sintácticos, semánticos y lógicos en fragmentos de código, e implementa estructuras de control de excepciones utilizando instrucciones propias del lenguaje para garantizar la estabilidad y el manejo adecuado de errores.
2. Analiza problemas y descompone en subproblemas, implementando funciones y procedimientos de manera efectiva en un lenguaje de programación.	IL2.1. Analiza problemas complejos y los descompone en subproblemas mediante representaciones como pseudocódigo o diagramas de flujo. IL2.2. Implementa funciones y procedimientos modulares aplicando paso de parámetros por valor y por referencia. IL2.3. Emplea correctamente variables locales y globales, garantizando el adecuado alcance y reutilización del código.
3. Implementa estructuras de datos, utilizando la sintaxis adecuada para representar y manipular información de manera eficiente en un lenguaje de programación.	IL3.1. Declara y manipula arreglos unidimensionales y bidimensionales mediante operaciones como lectura, escritura y actualización. IL3.2. Aplica operaciones básicas sobre cadenas de caracteres, como búsqueda, extracción y concatenación. IL3.3. Utiliza estructuras dinámicas simples como pilas, colas o listas para resolver problemas básicos de gestión de datos.
4. Implementa algoritmos de búsqueda y ordenamiento en estructuras de datos básicos, en un lenguaje de programación según el contexto y necesidades específicas.	IL4.1. Implementa algoritmos de búsqueda lineal y binaria en arreglos, comparando su eficiencia según el caso. IL4.2. Programa y compara algoritmos de ordenamiento interno como burbuja, inserción y selección, explicando su uso adecuado. IL4.3. Evalúa el rendimiento de los algoritmos implementados mediante métricas básicas como tiempo de ejecución o cantidad de comparaciones.



IV. Contrato didáctico

Estimados y estimadas estudiantes:

Con base en el Reglamento de Régimen Académico y Disciplinario de Grado (2024) y las disposiciones estipuladas por la Universidad Americana (UAM), a continuación, se detallan normas, responsabilidades y compromisos entre el docente y los estudiantes a fin de garantizar un proceso de enseñanza-aprendizaje efectivo para la asignatura Programación Estructurada.

a. UAM-Virtual

UAM-Virtual es el Entorno Virtual de Aprendizaje, es decir la plataforma en la que estudiantes y docente se apoyarán exclusivamente para el desarrollo de las actividades académicas. Es fundamental que cada estudiante haga uso adecuado de la plataforma para realizar las actividades académicas que corresponden al curso, dado que:

- El registro y control de asistencia será realizado mediante una actividad en el aula virtual.
- El registro de acumulados será realizado mediante la hoja de calificaciones en el aula virtual (notas oficiales en UAM Class)
- Las consultas al docente se realizarán mediante el Foro de Consultas o mediante videollamadas en el aula virtual
- La entrega de tareas se realizará únicamente por medio del aula virtual, a menos que el docente lo indique de otra manera
- Todo el material y herramientas a utilizarse en el curso estarán disponibles para descarga en el aula virtual

b. Honestidad académica

- Todos los trabajos académicos realizados por los estudiantes deben ser originales, evitando incurrir en plagio académico; para ello se deberá hacer uso del estilo APA (7ma Ed.) para referenciar y citar fuentes bibliográficas, cuando sea pertinente.
- Los estudiantes deberán evitar el fraude académico en todo sentido, el cual será penalizado con la pérdida de la calificación asignada a trabajos académicos, tanto individuales como grupales.
- Los entregables del curso deberán ser presentados en formato digital, por tanto, es responsabilidad de los estudiantes garantizar el acceso, disponibilidad, versión y buen estado de los archivos alojados en las diferentes plataformas: UAM-Virtual, Drive, GitHub, entre otros.

c. Uso de la Inteligencia Artificial

- Las herramientas de Inteligencia Artificial (IA), incluida la Inteligencia Artificial Generativa, podrán utilizarse únicamente cuando su uso sea compatible con las orientaciones establecidas para cada actividad académica. Estas herramientas constituyen recursos de apoyo al aprendizaje y no sustituyen la producción intelectual, el análisis, la toma de decisiones ni la responsabilidad del estudiante sobre el trabajo presentado. En



consecuencia, todo producto académico deberá reflejar el aprendizaje y las competencias desarrolladas por el estudiante, quien deberá comprender, verificar y ser capaz de explicar los contenidos, procedimientos o códigos que presente. Cuando el uso de IA sea permitido, el estudiante deberá utilizarla de manera ética y responsable, verificar críticamente sus resultados y declarar su utilización cuando así se solicite.

- La presentación como propia de una producción generada sustancialmente por IA, cuando esta sustituya el trabajo intelectual que corresponde realizar al estudiante o cuando su uso no haya sido autorizado, será considerada una práctica contraria a la integridad académica y podrá dar lugar a la aplicación de la reglamentación institucional correspondiente, incluyendo la posible afectación de la calificación de la actividad o evaluación.
- En las actividades de programación, el uso de IA no exime al estudiante de comprender el algoritmo, la lógica, las estructuras de datos, las funciones y el código presentado. El docente podrá solicitar al estudiante que explique, modifique, depure o reconstruya una solución como evidencia de su dominio individual

d. Normas de comportamiento en clase

- Asistir mínimamente al 80% de las sesiones de clases, de lo contrario perderá derecho a una calificación final.
- Asistir puntualmente al aula de clases, y contar con todos los materiales de apoyo y herramientas necesarios para su aprendizaje, así como participar activamente en las actividades de cada sesión de clases.
- Mantener una actitud respetuosa con sus compañeros, compañeras, docentes y la comunidad universitaria en general, así como el cuidado de las instalaciones y equipamiento propiedad de la Universidad Americana (UAM).
- No se admite en el aula de clases y laboratorios el ingreso de alimentos, bebidas, uso de celulares, medios de entretenimiento o computadoras personales para fines ajenos a la clase.
- Los únicos medios de comunicación aceptados entre docentes y estudiantes serán los medios establecidos en la plataforma UAM-Virtual y el correo electrónico docente y estudiantil respectivamente: @uamv.edu.ni



V. Planificación del contenido del curso

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
UNIDAD I: Validación y manejo de errores				
Sesión 1	17 de ago	Introducción a Python: Origen Versiones Características y usos Entorno de desarrollo de Python Usando el intérprete de Python	Conferencia dialogada Presentación de la asignatura: el docente presenta el syllabus de la asignatura, sus objetivos, contenidos y formas de evaluación Resolución de Cuestionario Prueba diagnóstica: los estudiantes responden un cuestionario en línea elaborado por el docente, el cual explora presaberes en estructuras de datos. Conferencia dialogada Introducción a Python: el docente expone generalidades del lenguaje de programación Python y activa presaberes entre los estudiantes. Se muestran de forma general algunos de los rankings de programación más populares para ubicar a Python entre otros lenguajes. Investigación Usos actuales de Python: los estudiantes presentan una lista de las principales aplicaciones de Python en el campo del desarrollo de software, realizando sus aportes en un foro. Práctica de laboratorio Instalación del entorno de desarrollo de Python: el docente explica los elementos del entorno de desarrollo de Python y explora su ubicación en la PC, dónde encontrarlos en la web. Instala y configura el entorno de desarrollo, según sea necesario, en conjunto con los estudiantes. Práctica en casa Instalación y configuración del entorno de desarrollo de Python: los estudiantes realizan la instalación y configuración del entorno de desarrollo de Python en sus computadoras personales.	Recursos del docente ▪ Syllabus de la asignatura ▪ Presentación Programación Estructurada ▪ Presentación Introducción a Python Herramientas ▪ UAM-Virtual Textos y material de lectura <i>Rankings sobre desarrollo de software</i> ▪ Stack Overflow (2025). Tecnología. https://survey.stackoverflow.co/2025/technology#most-popular-technologies-language-language-prof ▪ Carbonnelle, P. (2025). PYPL Popularity of Programming Language index. https://pypl.github.io/PYPL.html ▪ TIOBE Software. (2026). TIOBE Index. https://www.tiobe.com/tiobe-index/ ▪ Python Software Foundation. (s. f.). Download Python. Python.org. https://www.python.org/downloads/ ▪ Microsoft. (s. f.). Download Visual Studio Code. Visual Studio Code. https://code.visualstudio.com/Download

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
Sesión 2	19 de ago	El lenguaje de programación Python: - Palabras reservadas - Operadores y expresiones - Comentarios en Python - Tipos de datos y variables - Instrucciones de entrada y salida	Conferencia dialogada El Lenguaje de programación Python: el docente presenta los elementos del lenguaje Python (palabras reservadas, operadores, comentarios, tipos de datos, entrada/salida, estructuras de control, módulos, etc.). realiza demostraciones y aplica los conceptos, mediante ejemplos sencillos.	Recursos del docente - Syllabus de la asignatura - Presentación El lenguaje de programación Python Herramientas - UAM-Virtual - Visual Studio Code 1.133.0 - Python 3.13.2 Textos y material de lectura - Cuevas, A. (2016). Capítulo 1: Introducción. En Python 3: Curso práctico (1ra Ed., pp. 15 – 38). RA-MA Editorial
Sesión 3	21 de ago		Práctica de laboratorio Tipos de datos, expresiones, entrada y salida: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados.	
			Lectura obligatoria: los estudiantes realizan una lectura del material proporcionado por el docente, replicando los ejercicios resueltos, en el caso que la lectura los presente.	- Cuevas, A. (2016). Capítulo 2: Empezando a programar. En Python 3: Curso práctico (1ra Ed., pp. 41 – 179). RA-MA Editorial - Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html
			Resolución de problemas Programación con Python: los estudiantes resuelven una lista de ejercicios propuesta por el docente donde podrán en práctica los conceptos y herramientas del lenguaje de programación Python	
Sesión 4	24 de ago	Estructuras de control de flujo: bifurcaciones y bucles	Conferencia dialogada El Lenguaje de programación Python: el docente presenta los elementos del lenguaje Python (palabras reservadas, operadores, comentarios, tipos de datos, entrada/salida, estructuras de control, módulos, etc.). realiza demostraciones y aplica los conceptos, mediante ejemplos sencillos.	Recursos del docente - Syllabus de la asignatura - Presentación El lenguaje de programación Python Herramientas - UAM-Virtual - Visual Studio Code 1.133.0 - Python 3.13.2 Textos y material de lectura - Cuevas, A. (2016). Capítulo 3: Elementos fundamentales de programación: instrucción condicional y bucles. En Python 3: Curso
			Práctica de laboratorio Control de flujo del programa: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados.	
			Lectura obligatoria: los estudiantes realizan una lectura del material proporcionado por el docente, replicando los ejercicios resueltos, en el caso que la lectura los presente	

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
Sesión 5	26 de ago		Resolución de problemas Programación con Python: los estudiantes resuelven una lista de ejercicios propuesta por el docente donde podrán en práctica los conceptos y herramientas del lenguaje de programación Python	práctico (1ra Ed., pp. 85 – 140). RA-MA Editorial Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html
Sesión 6	28 de ago	Excepciones comunes en distintos lenguajes de programación Mecanismos de manejo de excepciones Estructuras de control: Uso de try, catch, finally para capturar y manejar excepciones.	Conferencia dialogada Validaciones y manejo de errores en Python: el docente presenta los mecanismos del lenguaje Python para realizar validaciones y el manejo de los errores en los programas, realiza demostraciones de los errores típicos y aplica los conceptos, mediante ejemplos sencillos. Práctica de laboratorio Validaciones y manejo de errores: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados. Lectura obligatoria: los estudiantes realizan una lectura del material proporcionado por el docente, replicando los ejercicios resueltos, en el caso que la lectura los presente Resolución de problemas Programación con Python: los estudiantes resuelven una lista de ejercicios propuesta por el docente donde podrán en práctica los conceptos y herramientas del lenguaje de programación Python	Recursos del docente - Syllabus de la asignatura - Presentación Validaciones y manejo de errores Herramientas - UAM-Virtual - Visual Studio Code 1.133.0 - Python 3.13.2 Textos y material de lectura - Cuevas, A. (2016). Capítulo 7: Ficheros y Excepciones. En Python 3: Curso práctico (1ra Ed., pp. 372 – 383). RA-MA Editorial - Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html
Sesión 7	31 de ago		Caza de bugs. El docente entrega a los estudiantes un programa con errores. Los estudiantes, en parejas, identifican, analizan y explican en tiempo real los bugs encontrados.	
Unidad II. Modularidad y funciones				
Sesión 8	2 de sep	Funciones internas Paquetes y módulos	Conferencia dialogada Modularidad y funciones en Python: el docente presenta las principales funciones internas del lenguaje, así como los mecanismos de paquete y módulo en Python para organizar programas considerando principios como: reutilización, escalabilidad, mantenibilidad.	Recursos del docente - Syllabus de la asignatura - Presentación Modularidad y Funciones en Python

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
			Práctica de laboratorio Funciones y módulos: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados. Lectura obligatoria: los estudiantes realizan una lectura del material proporcionado por el docente, replicando los ejercicios resueltos, en el caso que la lectura los presente Resolución de problemas Funciones en Python: los estudiantes resuelven una lista de ejercicios propuesta por el docente donde podrán en práctica los conceptos y herramientas del lenguaje de programación Python	Herramientas ▪ UAM-Virtual ▪ Visual Studio Code 1.133.0 ▪ Python 3.13.2 Textos y material de lectura ▪ Cuevas, A. (2016). Capítulo 2: Empezando a programar. En Python 3: Curso práctico (1ra Ed., pp. 41 – 179). RA-MA Editorial ▪ Cuevas, A. (2016). Capítulo 4: Programación funcional. En Python 3: Curso práctico (1ra Ed., pp. 145 – 179). RA-MA Editorial ▪ Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html
Sesión 9	4 de sep	Definición de subprograma Técnica de Divide y Vencerás	Conferencia dialogada Modularidad y funciones en Python: el docente explica la técnica de divide y vencerás para la organización de un programa en subprogramas aplicando los principios de: reutilización, escalabilidad, mantenibilidad. Práctica de laboratorio Modularidad: con la guía del docente, los estudiantes aplican la técnica de Divide y Vencerás para diseñar soluciones basadas en subprogramas o funciones. Se recomienda el uso del diagrama de cuadros para representar la estructura de subprogramas.	
Sesión 10	7 de sep	Funciones y procedimientos en Python	Sesion demostrativa. El docente expone el concepto de subprograma, detallando las diferencias entre funciones y procedimientos, y explicando los criterios para seleccionar su uso según el tipo de tarea y el resultado esperado. Práctica demostrativa de programación modular en Python. El docente resuelve con los estudiantes un problema cuya solución es modular, atendiendo principios de la técnica Divide y Vencerás.	Recursos del docente ▪ Syllabus de la asignatura ▪ Presentación Modularidad y Funciones en Python Herramientas ▪ UAM-Virtual ▪ Visual Studio Code 1.133.0 ▪ Python 3.13.2 Textos y material de lectura ▪ Cuevas, A. (2016). Capítulo 4: Programación funcional. En Python 3: Curso práctico (1ra Ed., pp. 145 – 179). RA-MA Editorial

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
				- Joyanes Aguilar, L. (2020). Capítulo 6: Subprogramas (subalgoritmos): Funciones. En Fundamentos de programación: Algoritmos, estructura de datos y objetos (5.ª ed., pp. 226-236). McGraw-Hill/Interamericana de España, S. A. U. Editorial. - Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html
Sesión 11	9 de sep	Ámbito de variables y funciones	Conferencia dialogada Modularidad y funciones en Python: el docente explica el concepto de ámbito de identificadores asociados a variables, objetos y funciones en el campo de la programación modular con uno y varios ficheros (archivos de código fuente). Actividad Diseña y Prueba. En parejas, los estudiantes diseñan un programa que tenga funciones que usen variables locales y, en el mismo programa, una variable global que sea modificada por distintos subprogramas. Deben mostrar el código al docente y explicar qué variables están disponibles en cada parte del programa y por qué.	Recursos del docente - Syllabus de la asignatura - Presentación Modularidad y Funciones en Python Herramientas - UAM-Virtual - Visual Studio Code 1.133.0 - Python 3.13.2 Textos y material de lectura - Cuevas, A. (2016). Capítulo 4: Programación funcional. En Python 3: Curso práctico (1ra Ed., pp. 145 – 179). RA-MA Editorial - Joyanes Aguilar, L. (2020). Capítulo 6: Subprogramas (subalgoritmos): Funciones. En Fundamentos de programación: Algoritmos, estructura de datos y objetos (5.ª ed., pp. 226-236). McGraw-Hill/Interamericana de España, S. A. U. Editorial. - Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html
Sesión 12	11 de sep	Paso de parámetros: por valor y por referencia.	Conferencia dialogada Modularidad y funciones en Python: el docente aborda el paso de parámetros en funciones y procedimientos, así como su tipificación en el lenguaje Python (objetos mutables, objetos inmutables) Clase demostrativa. Mediante un ejercicio práctico, el docente explica el paso de parámetros por valor y por referencia a las funciones. Práctica. Los estudiantes realizan una serie de ejercicios propuesto por el docente, aplicando de forma correcta el paso de parámetros por valor y por referencia	- Cuevas, A. (2016). Capítulo 4: Programación funcional. En Python 3: Curso práctico (1ra Ed., pp. 145 – 179). RA-MA Editorial - Joyanes Aguilar, L. (2020). Capítulo 6: Subprogramas (subalgoritmos): Funciones. En Fundamentos de programación: Algoritmos, estructura de datos y objetos (5.ª ed., pp. 226-236). McGraw-Hill/Interamericana de España, S. A. U. Editorial. - Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
Sesión 13	14 de sep	Aplicación de funciones y procedimientos	Resolución a problema. Conformados en equipos, los estudiantes resuelven un problema que integra todos los mecanismos de la programación modular. El caso a resolver es indicado por el docente.	Recursos del docente - Syllabus de la asignatura - Presentación Modularidad y Funciones en Python Herramientas - UAM-Virtual - Visual Studio Code 1.133.0 - Python 3.13.2 Textos y material de lectura - Cuevas, A. (2016). Capítulo 4: Programación funcional. En Python 3: Curso práctico (1ra Ed., pp. 145 – 179). RA-MA Editorial - Joyanes Aguilar, L. (2020). Capítulo 6: Subprogramas (subalgoritmos): Funciones. En Fundamentos de programación: Algoritmos, estructura de datos y objetos (5.ª ed., pp. 226-236). McGraw-Hill/Interamericana de España, S. A. U. Editorial. - Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html
Sesión 14	16 de sep			
Sesión 15	18 de sep		Exposición Presentación de solución. Los equipos presentan el diagrama de cuadros y el código fuente desarrollado en la sesión anterior, explicando cómo su solución aborda la problemática y la implementación de subprogramas y funciones.	
Sesión 16	21 de sep	Examen del 1er Corte Evaluativo.	Examen Los estudiantes resuelven un examen teórico práctico en el laboratorio, diagramando la estructura de módulos y codificando la solución a problemas planteados por el docente, aplicando modularidad en la construcción de soluciones con sintaxis del lenguaje Python.	
Unidad III: Estructuras de Datos				
Sesión 17	23 de sep	Arreglos unidimensionales: implementación mediante Listas de Python	Conferencia dialogada El docente introducirá la estructura de datos Lista, explicando su sintaxis en Python, la forma de declarar, asignar y acceder a sus elementos, así como las principales operaciones de lectura, escritura, asignación y actualización. Se presentarán ejemplos conceptuales en la pizarra y se discutirán casos prácticos donde los arreglos resulten una herramienta eficiente para almacenar y manipular datos de manera estructurada.	Recursos del docente - Syllabus de la asignatura - Presentación Modularidad y Funciones en Python Herramientas - UAM-Virtual - Visual Studio Code 1.133.0

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
			Demostración práctica El docente desarrollará un ejemplo en Python para declarar una Lista de tipo entero, asignar valores manualmente y mostrar su contenido con un ciclo for. Posteriormente, guiará a los estudiantes para modificar el código y permitir la entrada de datos por teclado, calcular la suma de los elementos y actualizar un valor específico.	- Python 3.13.2 Textos y material de lectura - Cuevas, A. (2016). Capítulo 6: Tipos de datos en Python. En Python 3: Curso práctico (1ra Ed., pp. 251 – 335). RA-MA Editorial
Sesión 18	25 de sep	Operaciones sobre listas: lectura, escritura, eliminación y actualización.	Clase práctica. Mediante la resolución de ejercicios propuestos por el docente, los estudiantes realizarán las operaciones de lectura, escritura, asignación y actualización sobre Listas.	Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html
Sesión 19	28 de sep	Arreglos bidimensionales: implementación mediante Listas de Listas en Python	Conferencia dialogada El docente presentará el concepto de listas anidadas explicando su sintaxis en Python, la forma de declarar, inicializar y asignar valores en filas y columnas, así como las operaciones básicas de lectura, escritura, asignación y actualización. Se utilizarán ejemplos teóricos para ilustrar la estructura en memoria de un arreglo bidimensional y se harán analogías con tablas de datos para facilitar la comprensión Demostración práctica El docente desarrollará un ejemplo en Python para declarar una lista anidada de tipo entero, asignar valores manualmente y mostrar su contenido con un ciclo for. Posteriormente, guiará a los estudiantes para modificar el código y permitir la entrada de datos por teclado, calcular la suma de los elementos y actualizar un valor específico de la lista.	Recursos del docente - Syllabus de la asignatura - Presentación Modularidad y Funciones en Python Herramientas - UAM-Virtual - Visual Studio Code 1.133.0 - Python 3.13.2 Textos y material de lectura - Cuevas, A. (2016). Capítulo 6: Tipos de datos en Python. En Python 3: Curso práctico (1ra Ed., pp. 251 – 335). RA-MA Editorial
Sesión 20	30 de sep	Operaciones con listas anidadas: lectura, escritura, eliminación y actualización.	Resolución de problemas Uso de listas: los estudiantes, guiados por el docente en el laboratorio de software, resolverán un reto que involucra el uso de listas anidadas en Python. Deberán crear un programa que permita declarar, asignar, leer, mostrar y actualizar datos en la lista, utilizando ciclos para recorrer sus elementos. El docente explicará cada paso clave, responderá dudas y propondrá variaciones del problema para que los estudiantes fortalezcan sus habilidades de programación y comprendan la utilidad de esta estructura de datos.	Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
Sesión 21	2 de oct	Cadenas de caracteres Representación de cadenas de caracteres en memoria. Operaciones básicas: búsqueda, concatenación, extracción y reemplazo.	Conferencia dialogada El docente presentará de forma sencilla cómo se representan las cadenas de caracteres en memoria y las operaciones básicas que se pueden realizar, como búsqueda, concatenación, extracción y reemplazo. Durante la explicación, se usarán ejemplos prácticos en Python y se formularán preguntas para que los estudiantes identifiquen posibles usos de cada operación, fomentando su participación y comprensión del tema. Práctica guiada En el laboratorio de software, los estudiantes desarrollarán pequeños programas en Python que permitan leer, escribir, asignar y actualizar cadenas, aplicando operaciones básicas como buscar una palabra en un texto o concatenar frases. El docente guiará paso a paso la práctica, proponiendo variaciones y retos cortos para que los estudiantes apliquen lo aprendido y afiancen sus habilidades en el manejo de datos de tipo cadena.	Recursos del docente - Syllabus de la asignatura - Presentación Cadenas de caracteres Herramientas - UAM-Virtual - Visual Studio Code 1.133.0 - Python 3.13.2 Textos y material de lectura - Cuevas, A. (2016). Capítulo 6: Tipos de datos en Python. En Python 3: Curso práctico (1ra Ed., pp. 251 – 335). RA-MA Editorial - Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html
Sesión 22	5 de oct	Archivos en Python	Conferencia dialogada Archivos en Python: el docente expone los mecanismos del lenguaje de programación Python para la persistencia de datos en archivos. Práctica guiada. En el laboratorio de software, los estudiantes desarrollarán pequeños programas en Python que permitan abrir, escribir, leer y guardar datos en un archivo.	Recursos del docente - Syllabus de la asignatura - Presentación Validaciones y manejo de errores Herramientas - UAM-Virtual - Visual Studio Code 1.133.0 - Python 3.13.2 Textos y material de lectura - Cuevas, A. (2016). Capítulo 7: Ficheros y Excepciones. En Python 3: Curso práctico (1ra Ed., pp. 372 – 383). RA-MA Editorial - Python Software Foundation. (2026). El tutorial de Python. Documentación de Python 3.13. https://docs.python.org/es/3.13/tutorial/index.html
Sesión 23	7 de oct	Operaciones sobre archivos: escribir, leer, modificar, eliminar	Resolución de problemas Uso de archivos: los estudiantes, guiados por el docente en el laboratorio de software, resolverán un reto que involucra los escenarios de guardar, recuperar, actualizar y eliminar registros de datos desde un archivo.	
Sesión 24	9 de oct	Implementación de persistencia con listas, cadenas y archivos	Caso de estudio Persistencia de datos: los estudiantes resuelven un caso de estudio en el que se utilizan listas y cadenas de caracteres para almacenar y recuperar datos hacia y desde archivos respectivamente.	

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
Sesión 25	12 de oct	Estructura de datos dinámica: Pila.	Conferencia dialogada Estructuras de datos lineales: el docente presenta los fundamentos de la estructura de datos lineal: Pila, realiza demostraciones y aplica los conceptos, mediante ejemplos sencillos. Práctica de laboratorio Pilas: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados. Simulación Operaciones sobre pilas: los estudiantes, formados en equipos, realizan la simulación gráfica de operaciones sobre listas enlazadas	Recursos del docente - Syllabus de la asignatura - Presentación Estructura de datos Pila Herramientas - UAM-Virtual - Visual Studio Code 1.133.0 - Python 3.13.2 Textos y material de lectura - Joyanes, L. (2008). Capítulo 12: Estructuras dinámicas lineales de datos (pilas, colas y listas enlazadas). En Fundamentos de Programación: Algoritmos, estructura de datos y objetos (4ta Ed., pp. 429 – 477). McGRAW-HILL
Sesión 26	14 de oct	Operaciones sobre Pilas.	Práctica de laboratorio Pilas: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados. Resolución de problemas Estructuras de datos lineales: los estudiantes resuelven una lista de ejercicios propuesta por el docente donde podrán implementar soluciones basadas en el uso de estructuras de datos lineales Exposición Simulación e implementación de las operaciones sobre Pilas: los estudiantes, conformados en equipos, presentan en plenario una simulación gráfica de las operaciones sobre pilas, y su implementación en Python	- Solución de problemas con algoritmos y estructuras de datos usando Python. https://runestone.academy/ns/books/publicshed/pythoned/index.html Herramientas - Visual Studio Code 1.98.0 - Python 3.13.2 - Procesador gráfico (Power Point, Canva, otros) Enlaces: - Simulador: <i>Visualising data structures and algorithms through animation</i>. https://visualgo.net/en
Sesión 27	16 de oct	Estructura de datos dinámica: Cola.	Conferencia dialogada Estructuras de datos lineales: el docente presenta los fundamentos de la estructura de datos lineal: Cola, realiza demostraciones y aplica los conceptos, mediante ejemplos sencillos.	Recursos del docente - Syllabus de la asignatura - Presentación Estructura de datos Cola

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
			Práctica de laboratorio Colas: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados. Simulación Operaciones sobre colas: los estudiantes, formados en equipos, realizan la simulación gráfica de operaciones sobre Colas	Textos y material de lectura Joyanes, L. (2008). Capítulo 12: Estructuras dinámicas lineales de datos (pilas, colas y listas enlazadas). En Fundamentos de Programación: Algoritmos, estructura de datos y objetos (4ta Ed., pp. 429 – 477). McGRAW-HILL
Sesión 28	19 de oct	Operaciones sobre Colas.	Práctica de laboratorio Colas: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados. Resolución de problemas Estructuras de datos lineales: los estudiantes resuelven una lista de ejercicios propuesta por el docente donde podrán implementar soluciones basadas en el uso de estructuras de datos lineales Exposición Simulación e implementación de las operaciones sobre Cola: los estudiantes, conformados en equipos, presentan en plenario una simulación gráfica de las operaciones sobre colas, y su implementación en Python.	- Solución de problemas con algoritmos y estructuras de datos usando Python. https://runestone.academy/ns/books/public/hed/pythoned/index.html Herramientas - Visual Studio Code 1.98.0 - Python 3.13.2 Enlaces: - Simulador: <i>Visualising data structures and algorithms through animation</i>. https://visualgo.net/en
Sesión 29	21 de oct	Estructura de datos dinámica: Lista Enlazada.	Conferencia dialogada Estructuras de datos lineales: el docente presenta los fundamentos de la estructura de datos lineal: Lista Enlazada, realiza demostraciones y aplica los conceptos, mediante ejemplos sencillos. Práctica de laboratorio Listas enlazadas: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados. Simulación Operaciones sobre listas: los estudiantes, formados en equipos, realizan la simulación gráfica de operaciones sobre listas enlazadas	Recursos del docente - Syllabus de la asignatura - Presentación Estructuras de datos lineales Textos y material de lectura - Joyanes, L. (2008). Capítulo 12: Estructuras dinámicas lineales de datos (pilas, colas y listas enlazadas). En Fundamentos de Programación: Algoritmos, estructura de datos y objetos (4ta Ed., pp. 429 – 477). McGRAW-HILL - Solución de problemas con algoritmos y estructuras de datos usando Python. https://runestone.academy/ns/books/public/hed/pythoned/index.html

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
Sesión 30	23 de oct	Operaciones sobre Listas Enlazadas	Práctica de laboratorio Listas enlazadas: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados. Resolución de problemas Estructuras de datos lineales: los estudiantes resuelven una lista de ejercicios propuesta por el docente donde podrán implementar soluciones basadas en el uso de estructuras de datos lineales Exposición Simulación e implementación de las operaciones sobre Lista: los estudiantes, conformados en equipos, presentan en plenario una simulación gráfica de las operaciones sobre listas enlazadas, y su implementación en Python	Herramientas - Visual Studio Code 1.98.0 - Python 3.13.2 - Procesador gráfico (Power Point, Canva, otros) Enlaces: - Simulador: <i>Visualising data structures and algorithms through animation</i>. https://visualgo.net/en
Sesión 31	26 de oct	Evaluación del 2do Corte	Examen Los estudiantes resuelven un examen teórico práctico en el laboratorio, codificando la solución a un problema planteado por el docente, en el que identificarán e implementarán una de las estructuras de datos estudiadas: Pilas, Colas, Listas Enlazadas, de acuerdo con las necesidades del problema.	
Sesión 32	28 de oct	Estructura de datos dinámica: Árbol	Conferencia dialogada Estructuras de datos no lineales: el docente presenta los fundamentos de la estructura de datos no lineales: Árbol y Árbol Binario, realiza demostraciones y aplica los conceptos, mediante ejemplos sencillos. Práctica de laboratorio Árboles: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados. Simulación Operaciones sobre árboles binarios: los estudiantes, formados en equipos, realizan la simulación gráfica de operaciones sobre árboles binarios	Recursos del docente - Syllabus de la asignatura - Presentación Estructuras de datos no lineales Textos y material de lectura - Joyanes, L. (2008). Capítulo 13: Estructuras de datos no lineales (árboles y grafos). En Fundamentos de Programación: Algoritmos, estructura de datos y objetos (4ta Ed., pp. 479 – 517). McGRAW-HIL

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
Sesión 33	30 de oct	Operaciones sobre árboles.	Práctica de laboratorio Árboles Binarios: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados Exposición Simulación e implementación de las operaciones sobre Cola: los estudiantes, conformados en equipos, presentan en plenario una simulación gráfica de las operaciones sobre colas, y su implementación en Python	- Solución de problemas con algoritmos y estructuras de datos usando Python. https://runestone.academy/ns/books/public/hed/pythoned/index.html Herramientas - Visual Studio Code 1.98.0 - Python 3.13.2 Enlaces: - Simulador: <i>Visualising data structures and algorithms through animation</i>. https://visualgo.net/en
Sesión 34	2 de nov	Estructura de datos dinámica: Grafo	Conferencia dialogada Estructuras de datos no lineales: el docente presenta los fundamentos de la estructura de datos no lineal: Grafo, realiza demostraciones y aplica los conceptos, mediante ejemplos sencillos. Práctica de laboratorio Grafos: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados	Recursos del docente - Syllabus de la asignatura - Presentación Estructuras de datos no lineales Textos y material de lectura - Joyanes, L. (2008). Capítulo 13: Estructuras de datos no lineales (árboles y grafos). En Fundamentos de Programación: Algoritmos, estructura de datos y objetos (4ta Ed., pp. 479 – 517). McGRAW-HIL
Sesión 35	4 de nov	Operaciones sobre grafos	Práctica de laboratorio Grafos: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar en la computadora ejercicios previamente seleccionados Exposición Aplicación de estructuras de datos no lineales: los estudiantes, organizados en equipos, exponen una solución que resuelve un problema determinado utilizando estructuras de datos no lineales	- Solución de problemas con algoritmos y estructuras de datos usando Python. https://runestone.academy/ns/books/public/hed/pythoned/index.html Herramientas - Visual Studio Code 1.98.0 - Python 3.13.2 Enlaces: - Simulador: <i>Visualising data structures and algorithms through animation</i>. https://visualgo.net/en

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
Sesión 36	6 de nov	Implementación de soluciones basadas en estructuras de datos dinámicas	Proyecto Integrador Estructuras de Datos Dinámicas: en esta actividad colaborativa, los equipos de estudiantes identificarán y resolverán un problema del mundo real diseñando una solución tecnológica que integre estructuras de datos dinámicas para el manejo eficiente de la información y el uso de inteligencia artificial generativa para la creación automatizada de interfaces gráficas intuitivas que permitan su visualización y manipulación, fomentando así el desarrollo de software completo, innovador y funcional.	Recursos del docente - Syllabus de la asignatura Textos y material de lectura - Joyanes, L. (2008). Capítulo 12: Estructuras dinámicas lineales de datos (pilas, colas y listas enlazadas). En Fundamentos de Programación: Algoritmos, estructura de datos y objetos (4ta Ed., pp. 429 – 477). McGRAW-HILL - Joyanes, L. (2008). Capítulo 13: Estructuras de datos no lineales (árboles y grafos). En Fundamentos de Programación: Algoritmos, estructura de datos y objetos (4ta Ed., pp. 479 – 517). McGRAW-HILL - Solución de problemas con algoritmos y estructuras de datos usando Python. https://runestone.academy/ns/books/publicshed/pythoned/index.html Herramientas - Visual Studio Code 1.98.0 - Python 3.13.2
Sesión 37	9 de nov	Implementación de soluciones basadas en estructuras de datos dinámicas		
Sesión 38	11 de nov	Implementación de soluciones basadas en estructuras de datos dinámicas		

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
Unidad IV. Algoritmos de búsqueda y ordenamiento				
Sesión 39	13 de nov	1. Análisis de la eficiencia de algoritmos 1.1 Orden algorítmico La Notación Big O 2. Algoritmos de búsqueda y ordenación Análisis del algoritmo de ordenamiento: Burbuja	Conferencia dialogada Análisis de algoritmos de ordenamiento y búsqueda: el docente presenta cómo se realiza el análisis de algoritmos, realiza demostraciones y aplica conceptos mediante ejemplos sencillos Práctica Identificación de los órdenes de algoritmos: los estudiantes realizan comparaciones de eficiencia a partir del análisis comparativo de funciones de la notación O grande. Caso de estudio Simulación de algoritmos de ordenamiento: los estudiantes, conformados en equipos de trabajo, diseñan una simulación gráfica del comportamiento de un algoritmo de ordenamiento asignado por el docente Práctica El docente caracteriza el algoritmo de Burbuja.	Recursos del docente - Syllabus de la asignatura - Presentación Análisis de eficiencia de algoritmos Textos y material de lectura - Joyanes, L. (2008). Capítulo 10: Ordenación, búsqueda e intercalación. En Fundamentos de Programación: Algoritmos, estructura de datos y objetos (4ta Ed., pp. 355 – 403). McGRAW-HILL - Solución de problemas con algoritmos y estructuras de datos usando Python. https://runestone.academy/ns/books/published/pythoned/index.html Herramientas - Visual Studio Code 1.98.0 - Python 3.13.2 - Hoja de cálculo (Excel / G Sheet) Enlaces: - Algorithms. + Visualizador https://www.hackerearth.com/practice/algorithms/sorting
Sesión 40	16 de nov	Análisis A priori 3. Cálculo de la eficiencia temporal de algoritmos de ordenación 3. Cálculo de la eficiencia espacial de algoritmos de ordenación	Conferencia dialogada El docente explica cómo se aplica en análisis asintótico de algoritmos para realizar evaluaciones y comparaciones entre ejemplares Práctica Análisis de eficiencia de algoritmos: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar ejercicios de cálculo de eficiencia temporal y espacial del algoritmo de ordenamiento Burbuja.	Recursos del docente - Syllabus de la asignatura - Presentación Análisis de Algoritmos de Ordenamiento y Búsqueda Textos y material de lectura - Joyanes, L. (2008). Capítulo 10: Ordenación, búsqueda e intercalación. En Fundamentos de Programación:

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
			Caso de estudio Análisis a priori: los estudiantes, conformados en equipos de trabajo, realizan el análisis algorítmico a priori, sobre un algoritmo de ordenamiento asignado por el docente	Algoritmos, estructura de datos y objetos (4ta Ed., pp. 355 – 403). McGRAW-HILL - Solución de problemas con algoritmos y estructuras de datos usando Python. https://runestone.academy/ns/books/public/hed/pythoned/index.html Herramientas - Visual Studio Code 1.98.0 - Python 3.13.2 - Hoja de cálculo (Excel / G Sheet) Enlaces: - Algorithms. + Visualizador https://www.hackerearth.com/practice/algorithms/sorting
Sesión 41	18 de nov	Análisis a posteriori	Práctica Análisis de eficiencia de algoritmos: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para replicar ejercicios de cálculo de eficiencia temporal y espacial del algoritmo de ordenamiento Burbuja. Caso de estudio Análisis a posteriori: los estudiantes, conformados en equipos de trabajo, realizan el análisis algorítmico a posteriori, sobre un algoritmo de ordenamiento asignado por el docente.	
Sesión 42	20 de nov	Análisis de algoritmos de búsqueda: Lineal y Binaria	Conferencia dialogada Análisis de algoritmos de ordenamiento y búsqueda: el docente aborda la importancia de la búsqueda en el procesamiento de datos e introduce los algoritmos de búsqueda: lineal y binaria, realizando una comparativa desde el punto de vista de la eficiencia. Simulación Con la ayuda de un simulador en línea, los estudiantes comprenderán el funcionamiento de los algoritmos de búsqueda lineal y binaria a través de una simulación. Práctica de laboratorio Implementación de algoritmos de búsqueda: los estudiantes, mediante el método hands-on learning o aprendizaje práctico, son guiados por el docente para implementar algoritmos de búsqueda	Recursos del docente - Syllabus de la asignatura - Presentación Análisis de Algoritmos de Ordenamiento y Búsqueda Textos y material de lectura - Joyanes, L. (2008). Capítulo 11: Ordenación, búsqueda y fusión externa. En Fundamentos de Programación: Algoritmos, estructura de datos y objetos (4ta Ed., pp. 405 – 427). McGRAW-HILL - Solución de problemas con algoritmos y estructuras de datos usando Python. https://runestone.academy/ns/books/public/hed/pythoned/index.html Herramientas - Visual Studio Code 1.98.0 - Python 3.13.2

Programación Orientada a Objetos				
Escenarios de Aprendizaje	Aula	Laboratorio	Extensión de Aula (con apoyo de recursos tecnológicos)	Institución Receptora
	C-202	Laboratorio de Computación	Plataforma Educativa UAM-Virtual	
Descripción		Contenidos	Estrategias de Aprendizaje	Recursos para el aprendizaje
Sesión 43	23 de nov	Revisión de resultados Evaluación de la eficiencia de algoritmos	Caso de estudio Evaluación de la eficiencia de algoritmos: los equipos de trabajo presentan avances de los resultados del análisis de eficiencia de algoritmos: simulación del algoritmo, implementación del algoritmo, definición asintótica y resultados del análisis a priori	
Sesión 44	25 de nov	Revisión de resultados Evaluación de la eficiencia de algoritmos	Caso de estudio Evaluación de la eficiencia de algoritmos: los equipos de trabajo presentan avances de los resultados del análisis de eficiencia de algoritmos: resultados del análisis a posteriori, demostración del comportamiento algorítmico según la notación O grande.	
Sesión 45	27 de nov	Evaluación del 3er Corte: Presentación de resultados Evaluación de la eficiencia de algoritmos	Caso de estudio Evaluación de la eficiencia de algoritmos: los equipos de trabajo realizan un resumen ejecutivo de los principales resultados del proceso de análisis de eficiencia de algoritmos y responden de manera individual a preguntas realizadas por el docente, donde demostrarán el dominio del proceso y los resultados obtenidos.	



VI. Mediación Pedagógica

La enseñanza se concibe como un proceso que impulsa el desarrollo de los estudiantes, que rescata la coherencia entre conocimiento y vida, que se centra en el reconocimiento de la singularidad de cada alumno, y en el rol protagónico que le corresponde desempeñar frente a su aprendizaje, comprendiéndose este último como el resultado de la actividad cognitiva y afectiva del estudiante en la que el docente asume un rol de acompañante, mediador, guía o facilitador, de forma que sus acciones sólo cobran sentido en la medida en que acompañan, apoyan y facilitan efectivamente el aprendizaje del estudiante.

Para la enseñanza de la asignatura Programación Estructurada se utilizará el aprendizaje experiencial, que facilita espacios para construir aprendizajes significativos desde la experimentación, utilizando el concepto hands-on learning o aprendizaje práctico en el laboratorio de software. Dada la naturaleza de los contenidos del plan temático se propone brindar a los estudiantes escenarios de experimentación que le facilite diseñar propuestas de solución que pueda someter a un análisis de comportamiento en el laboratorio de software.

Puesto que el aprendizaje en esta asignatura es visto como un proceso de descubrimiento, de búsqueda de alternativas, de solución de problemas, son ejes fundamentales de dicho proceso: aprender a aprender, la investigación, la práctica, la búsqueda de significados, el establecimiento de vínculos entre lo que ya se sabe y lo nuevo por conocer, el aprender de otros y con otros.

En correspondencia, para el aprendizaje de la asignatura Programación Estructurada será necesario un conjunto de estrategias prácticas para el estudio individual y grupal, dentro de las cuales se recomiendan:

- Prácticas de laboratorio
- Investigaciones
- Resolución de problemas
- Lecturas seleccionadas
- Evaluaciones prácticas
- Casos de estudio
- Simulaciones gráficas
- Exposiciones

Por tratarse de una asignatura de formación específica que sirve de base para futuras asignaturas de talante tecnológico, se requiere de un acompañamiento cercano del docente para identificar y remediar los problemas de aprendizaje individual y grupal, así como los relacionados con la enseñanza misma.

VII. Evaluación para el aprendizaje

Objetivos de aprendizaje	Indicadores de logro (IL)	Técnicas de evaluación	Instrumento (define los criterios de la evaluación)	Tipo de evaluación	Fecha	Puntaje (0 a 100)
Primer Corte Evaluativo (Del 21 al 25 de septiembre)						
Objetivos de aprendizaje 1 y 2	IL 1.1	Investigación Usos actuales de Python	Respuesta en foro	Formativa	19 de sep	-
		Resolución de problemas Programación con Python	Lista de cotejo	Formativa	Del 19 al 28 de ago	-
		Práctica de laboratorio Validaciones y manejo de errores	Rúbrica	Formativa	28 de ago	-
		Caza de bugs	Lista de cotejo	Sumativa	31 de ago	30
	IL2.1, IL2.2	Resolución de problemas Funciones en Python	Lista de cotejo	Formativa	Del 2 al 18 de sep	-
		Exposición Solución modular con Python	Rúbrica	Sumativa	16 y 18 de sep	30
	IL2.1, IL2.2, IL2.3	Evaluación del 1er Corte	Rúbrica	Sumativa	21 de sep	40
Segundo Corte Evaluativo (Del 26 al 30 de octubre)						
Objetivos de aprendizaje 3 y 4	IL3.1	Resolución de problemas Uso de listas	Lista de cotejo	Formativa	Del 23 al 30 de sep	-
	IL3.2	Resolución de problemas Cadenas de caracteres	Lista de cotejo	Formativa	Del 2 al 5 de oct	-
	IL3.1 y 3.2	Implementación de persistencia con listas, cadenas y archivos	Rúbrica	Sumativa	9 de oct	30
		Simulación Operaciones sobre pilas, colas y listas	Lista de cotejo	Formativa	Del 12 al 23 de oct	-
	IL3.3	Resolución de problemas Estructuras de datos lineales	Lista de cotejo	Sumativa	Del 12 al 23 de oct	30
		Evaluación del 2do Corte	Rúbrica	Sumativa	26 de oct	40
Tercer Corte Evaluativo (Del 7 al 11 de diciembre)						
Objetivos de aprendizaje 3 y 4	IL3.3	Simulación Operaciones sobre árboles y grafos	Lista de cotejo	Formativa	Del 28 de oct al 4 de nov	-
		Exposición Aplicación de estructuras de datos no lineales	Lista de cotejo	Sumativa	Del 28 de oct al 4 de nov	20
		Proyecto Integrador Estructuras de Datos Dinámicas	Rúbrica	Sumativa	Del 6 al 11 de nov	40
	IL4.2	Caso de estudio Simulación de algoritmos de ordenamiento	Lista de cotejo	Formativa	13 de nov	-
		Caso de estudio Análisis a priori	Lista de cotejo	Formativa	16 de nov	-
		Caso de estudio Análisis a posteriori	Lista de cotejo	Formativa	18 de nov	-
	IL4.1	Práctica de laboratorio Implementación de algoritmos de búsqueda	Lista de cotejo	Formativa	20 de nov	-
	IL4.3	Evaluación del 3er Corte	Rúbrica	Sumativa	27 de nov	40
NOTA FINAL						100
Primer corte Evaluativo						100
Segundo corte evaluativo						100
Tercer corte evaluativo						100
NOTA FINAL: (I C + II C + III C) / 3						100



Matriz informativa para el nivel de desempeño logrado de acuerdo al puntaje alcanzado.

Nivel Cualitativo	Equivalencia Cuantitativa	Descriptor
Aprendizaje Inicial	0 - 69 puntos	Dominio limitado o en desarrollo: evidencia habilidades y destrezas mínimas o parciales, con dificultad para aplicarlas en contextos reales.
Aprendizaje Fundamental	70 - 79 puntos	Dominio básico: demuestra comprensión fundamental de conceptos y habilidades, aunque con limitaciones en su integración y aplicación contextual.
Aprendizaje Satisfactorio	80 - 89 puntos	Dominio adecuado: aplica habilidades y conocimientos de manera integrada en situaciones reales, cumpliendo con los criterios establecidos.
Aprendizaje Avanzado	90 - 100 puntos	Dominio sobresaliente: evidencia manejo excepcional de las capacidades, con aplicación innovadora y transferencia creativa del conocimiento.



VIII. Referencias bibliográficas

Obligatorias

Cuevas, A. (2016). Python 3. Curso Práctico. RA-MA Editorial

Cairo, O., Gardati, S. (2011) Estructuras de Datos. McGraw Hill, México, D.F.

Optativas

Python Software Foundation (2026). El tutorial de Python.
<https://docs.python.org/es/3.13/tutorial/index.html>

Joyanes, L. (2008). Fundamentos de Programación: Algoritmos, estructura de datos y objetos. McGRAW-HILL/INTERAMERICANA DE ESPAÑA, S. A. U.

Herramientas de software

Git [Software]. Git SCM. <https://git-scm.com/downloads>

GitHub. (2026). *GitHub* [Software]. <https://github.com/>

HackerEarth. (s. f.). *Sorting*. HackerEarth.
<https://www.hackerearth.com/practice/algorithms/sorting>

Microsoft. (2026). *Visual Studio Code* [Software]. <https://code.visualstudio.com/download>

Python Software Foundation. (2025). *Python (Versión 3.13)* [Software].
<https://www.python.org/downloads/release/python-3130/>

VisuAlgo. (2011.). *VisuAlgo: Visualising data structures and algorithms through animation*.
<https://visualgo.net/en>